

PRACTICAL - 9

TASK1: Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

```
root@DESKTOP-QU0HVJI: ~$ mkdir Practical9
root@DESKTOP-QU0HVJI: ~$ cd Practical9
root@DESKTOP-QU0HVJI: ~/Practical9$ nano lowlevel_copy.c
root@DESKTOP-QU0HVJI: ~/Practical9$ gcc lowlevel_copy.c -o lowlevel_copy
root@DESKTOP-QU0HVJI: ~/Practical9$ ls
lowlevel_copy  lowlevel_copy.c
root@DESKTOP-QU0HVJI: ~/Practical9$ echo "This is my Week 9 Linux file I/O practical." > input.txt
root@DESKTOP-QU0HVJI: ~/Practical9$ cat input.txt
This is my Week 9 Linux file I/O practical.
root@DESKTOP-QU0HVJI: ~/Practical9$ ./lowlevel_copy input.txt copied_lowlevel.txt
File copied successfully.
Bytes copied: 44
root@DESKTOP-QU0HVJI: ~/Practical9$ cat copied_lowlevel.txt
This is my Week 9 Linux file I/O practical.
root@DESKTOP-QU0HVJI: ~/Practical9$ cmp input.txt copied_lowlevel.txt
root@DESKTOP-QU0HVJI: ~/Practical9$ |
```

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main(int argc, char *argv[])
{
    if (argc != 3)
    {
        printf("Usage: %s <source> <destination>\n", argv[0]);
        return 2;
    }

    // Open source file in read-only mode
    int src = open(argv[1], O_RDONLY);

    if (src == -1)
    {
        perror("Error opening source file");
        return 2;
    }

    // Use lseek() to find the size of the source file
    off_t file_size = lseek(src, 0, SEEK_END);

    if (file_size == (off_t)-1)
    {
        perror("Error using lseek");
        close(src);
        return 2;
    }

    // Move file pointer back to the beginning
    if (lseek(src, 0, SEEK_SET) == (off_t)-1)
    {
        perror("Error resetting file position");
        close(src);
    }
}
```

```
root@DESKTOP-QU0HVJI: ~/f × + v
GNU nano 7.2 lowlevel_copy.c
    close(src);
    return 1;
}

// Open/create destination file
int dest = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);

if (dest == -1)
{
    perror("Error opening destination file");
    close(src);
    return 2;
}

char buffer[4096];
ssize_t bytes_read;

// Read and write until the whole file is copied
while ((bytes_read = read(src, buffer, sizeof(buffer))) > 0)
{
    ssize_t total_written = 0;

    while (total_written < bytes_read)
    {
        ssize_t bytes_written =
            write(dest,
                buffer + total_written,
                bytes_read - total_written);

        if (bytes_written == -1)
        {
            perror("Error writing file");
            close(src);
            close(dest);
            return 1;
        }
    }
}

if (bytes_read == -1)
{
    perror("Error reading file");
}

// Close both files
close(src);
close(dest);

printf("File copied successfully.\n");
printf("Bytes copied: %lld\n", (long long)file_size);

return 0;
}
```

TASK2: Develop a program that redirects standard input and output using file descriptors and dup2(). Demonstrate how shells implement I/O redirection.

```
root@DESKTOP-QU0HVJI:~/Practical9$ nano stdlib_copy.c
root@DESKTOP-QU0HVJI:~/Practical9$ gcc stdlib_copy.c -o stdlib_copy
root@DESKTOP-QU0HVJI:~/Practical9$ ./stdlib_copy input.txt copied_stdlib.txt
File copied successfully using standard library functions.
root@DESKTOP-QU0HVJI:~/Practical9$ cat copied_stdlib.txt
This is my Week 9 Linux file I/O practical.
root@DESKTOP-QU0HVJI:~/Practical9$ cp input.txt copied_stdlib.txt
root@DESKTOP-QU0HVJI:~/Practical9$ dd if=/dev/zero of=largefile.bin bs=1M count=100 status=progress
100+0 records in
100+0 records out
104857600 bytes (105 MB, 100 MiB) copied, 0.0799219 s, 1.3 GB/s
root@DESKTOP-QU0HVJI:~/Practical9$ time ./lowlevel_copy largefile.bin lowlevel_result.bin
File copied successfully.
Bytes copied: 104857600

real    0m0.117s
user    0m0.000s
sys     0m0.110s
root@DESKTOP-QU0HVJI:~/Practical9$ time ./stdlib_copy largefile.bin stdlib_result.bin
File copied successfully using standard library functions.

real    0m0.131s
user    0m0.009s
sys     0m0.122s
root@DESKTOP-QU0HVJI:~/Practical9$ cmp largefile.bin lowlevel_result.bin
root@DESKTOP-QU0HVJI:~/Practical9$ cmp largefile.bin stdlib_result.bin
root@DESKTOP-QU0HVJI:~/Practical9$
```

```
root@DESKTOP-QU0HVJI: ~/t × + ▾
GNU nano 7.2                                                                    stdlib_copy.c *
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    if (argc != 3)
    {
        printf("Usage: %s <source> <destination>\n", argv[0]);
        return 1;
    }

    // Open source file for reading
    FILE *src = fopen(argv[1], "rb");

    if (src == NULL)
    {
        perror("Error opening source file");
        return 1;
    }

    // Open destination file for writing
    FILE *dest = fopen(argv[2], "wb");

    if (dest == NULL)
    {
        perror("Error opening destination file");
        fclose(src);
        return 1;
    }

    char buffer[4096];
    size_t bytes_read;

    // Read using fread() and write using fwrite()
    while ((bytes_read = fread(buffer, 1, sizeof(buffer), src)) > 0)
    {
        if (fwrite(buffer, 1, bytes_read, dest) != bytes_read)
```